Programming Assignment: Tomasulo Algorithm with Multi-Issue

Dipta Chandra Paul, COSC 6385 Computer Architecture

1. Introduction

The starting point was the provided Python simulator of the Tomasulo algorithm (register renaming with a RAT, ROB, reservation stations, load/store queue with memory disambiguation). It issues and commits one instruction per cycle. In this assignment I

1. added **multi-issue support**: up to 1–4 instructions are issued per cycle **in program order**, and the same number can commit per cycle (also in order). The issue width is read from the input file (`Issue width = N`, default 1);
2. made the simulator read the **whole input file** in the format of the assignment (hardware configuration, ROB size, issue width, initial registers and memory, program) and print the **register and non-zero memory values** at the end;
3. fixed several bugs in the provided code that showed up once more than one instruction was in flight per cycle;
4. wrote 10 test cases and an automatic checker that tests every run for correctness, and ran all test cases with issue width 1, 2, 3 and 4.

2. Brief description of what I added to the code

Multi-issue, in order (`issue.py`). The old one-instruction issue function is now `issue_one()`, which returns `True` when the instruction was issued. The new `issue()` calls it up to `issue_width` times per cycle:

```
def issue(cycle, PC, instructions, ROB, size_ROB,
        rs_int_adder,
        rs_fp_adder,
        rs_fp_multi,
        ld_sd_queue, size_ld_sd_queue,
        rat_int, rat_fp, issue_width=1):
    nIssued = 0

    while (nIssued < issue_width) & (PC.valid == 1) & (PC.PC < len(instructions)):
        isIssued = issue_one(cycle, PC, instructions, ROB, size_ROB,
                             rs_int_adder, rs_fp_adder, rs_fp_multi,
                             ld_sd_queue, size_ld_sd_queue, rat_int, rat_fp)

        # in order: if this instruction has no ROB/RS/queue space,
        # the younger instructions cannot be issued either
        if not isIssued:
            break

        nIssued += 1
        # after a branch PC.valid is 0, so the loop stops until the branch is resolved

    return nIssued
```

- Issuing stops at the first instruction that cannot be issued (ROB, reservation station or load/store queue full) and after a branch, so instructions always issue in program order.
- Instructions issued in the same cycle are renamed one after the other, so a dependence inside the group gets the correct ROB tag.

The other changes:

- **Multi-commit, in order (`commit.py`).** Up to `issue_width` finished instructions leave the head of the ROB per cycle, stopping at the first unfinished one.
- **Issue width from the input file (`init.py`, `main.py`).** The simulator now reads the whole input file in the assignment format, including `Issue width = N` (1–4, default 1). At the end it prints the status table, the registers and the non-zero memory values. `python3 main.py <input file> [issue width]` can override the width.
- **Bug fixes.** Several bugs in the provided code appeared once more than one instruction was in flight; the most important was the RAT being overwritten by an older result. These, and the other details of the changes, are in Appendix A.

3. Test cases

3.1 How correctness is checked

`code/run_tests.py` runs every test case in `tests/` with issue width 1, 2, 3 and 4, saves each output in `results/<test>_w<N>.txt`, and checks every run with `code/check.py`:

1. **Functional check.** A simple reference model executes the same program one instruction at a time (no pipeline). The simulator's final registers and memory and the number of executed instructions must be exactly the same.
2. **Timing check.**
    - No more than `issue width` instructions are issued or committed in any cycle.
    - Issue and commit happen in program order.
    - For every instruction: ISSUE < EX ≤ (MEM) < WB < COMMIT.
    - At most one instruction per CDB writes back in a cycle.
    - Memory accesses never overlap (single port).
    - No instruction after a branch is issued before the branch is resolved.

To make sure the checker really finds errors, I temporarily put the old RAT bug back in `wb.py`. `run_tests.py` then reported 12 failed runs: `tc7_branches`, `tc8_sample_report_case1` and `tc9_sample_report_case2` failed at every issue width because of wrong FP registers (and, for `tc7_branches`, wrong memory). With the fix, all runs pass.

3.2 Test cases and total cycles

All 40 runs (10 test cases × 4 issue widths) pass both checks. Total number of cycles (cycle of the last commit) for each issue width W:

Test case	What it tests	W=1	W=2	W=3	W=4
`tc1_given_loop`	Test case provided with the code (loop with Bne, loads, store, FP ops); branch offset written as -3 (instructions).	41	36	35	34
`tc2_independent`	12 independent integer/FP instructions, 2 FUs per type, one CDB.	19	15	15	15
`tc3_independent_4cdb`	Same program as tc2 but with the optional `CDB = 4` (removes the CDB bottleneck).	19	13	11	10

Test case	What it tests	W=1	W=2	W=3	W=4
`tc4_dependences`	RAW chains, WAW and WAR on F4/F1/R2 (register renaming), structural stalls on the FP adder RS, write to R0.	32	28	27	26
`tc5_memory`	Store-to-load forwarding, loads waiting for unknown store addresses, loads bypassing stores with different addresses.	25	25	25	25
`tc6_structural`	Very small machine (ROB = 4, 1 RS per unit, 2 load/store entries): in-order issue must stall.	29	29	29	29
`tc7_branches`	Loop with Bne (3 iterations), taken/not-taken Beq, instructions skipped by branches.	34	27	24	24
`tc8_sample_report_case1`	Test case #01 from the sample report handed out with the assignment.	36	35	35	35
`tc9_sample_report_case2`	Test case #02 from the sample report (2 FP adders, 2 FP multipliers, 2 address adders).	49	47	47	47
`tc10_pdf_format`	Configuration and syntax of the assignment PDF (commas, lower case, `Issue width = 2`, Bne loop, negative offset in Sd).	41	37	37	37

4. Results of test cases under different issue widths

4.1 Discussion

- **Issue really is wider.** In every table below, up to W instructions get the same ISSUE cycle and up to W instructions get the same COMMIT cycle, always in program order. Instructions that used to wait to be fetched now enter the reservation stations earlier and can start as soon as their operands are ready.
- **Speedup is largest from 1 to 2.** For example, tc1 goes from 41 to 36 cycles, tc7 from 34 to 24 cycles at width 3, and tc4 from 32 to 26 cycles at width 4. After that, other limits dominate.
- **One CDB.** At most one result can be written back per cycle, so a machine that issues 4 instructions per cycle can still only complete 1 per cycle. tc2 and tc3 run the same 12 independent instructions. With one CDB (tc2), widths 2–4 all need 15 cycles. With `CDB = 4` (tc3), the time keeps dropping: 19 → 13 → 11 → 10 cycles.
- **Branches.** The machine has no branch prediction (as in the provided code), so fetching stops at every branch until it is resolved. In tc1 the loop body after the first iteration is only 3 instructions with a branch at the end, so a width above 2 helps very little: widths 2, 3 and 4 need 36, 35 and 34 cycles.
- **Memory and long latencies.** tc5 is limited by the single-ported memory (25 cycles for every width). tc8/tc9 are limited by long FP-multiply chains and the memory latency.
- **Structural hazards.** In tc6 the ROB has only 4 entries, so wider issue gives no gain (29 cycles for every width). The tables show issue stopping when a reservation station or the ROB is full.

4.2 Given test case (tc1) with issue width 1, 2, 3 and 4

tc1 (given test case), issue width 1

	ISSUE	EX	MEM	WB	COMMIT
Ld F2, 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F4, F2, F20	[2]	[9, 23]	[]	[24]	[25]
Ld F6, 0(R2)	[3]	[4, 4]	[8, 12]	[13]	[26]
Add.d F6, F4, F6	[4]	[25, 28]	[]	[29]	[30]
Sd F6, 0(R2)	[5]	[6, 6]	[31, 35]	[]	[31]
Addi R1, R1, -4	[6]	[7, 7]	[]	[9]	[32]
Addi R2, R2, -4	[7]	[8, 8]	[]	[10]	[33]
Bne R1, R0, -3	[8]	[10, 10]	[]	[]	[34]
Addi R1, R1, -4	[11]	[12, 12]	[]	[14]	[35]
Addi R2, R2, -4	[12]	[13, 13]	[]	[15]	[36]
Bne R1, R0, -3	[13]	[15, 15]	[]	[]	[37]
Addi R1, R1, -4	[16]	[17, 17]	[]	[18]	[38]
Addi R2, R2, -4	[17]	[18, 18]	[]	[19]	[39]
Bne R1, R0, -3	[18]	[19, 19]	[]	[]	[40]
Add.d F20, F2, F2	[20]	[21, 24]	[]	[25]	[41]

Final non-zero registers: R2=20, F2=1.0, F4=3.0, F6=7.0, F20=2.0
Non-zero memory: Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=7.0

tc1 (given test case), issue width 2

	ISSUE	EX	MEM	WB	COMMIT
Ld F2, 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F4, F2, F20	[1]	[9, 23]	[]	[24]	[25]
Ld F6, 0(R2)	[2]	[3, 3]	[8, 12]	[13]	[25]
Add.d F6, F4, F6	[2]	[25, 28]	[]	[29]	[30]
Sd F6, 0(R2)	[3]	[4, 4]	[31, 35]	[]	[31]
Addi R1, R1, -4	[3]	[4, 4]	[]	[5]	[32]
Addi R2, R2, -4	[4]	[5, 5]	[]	[6]	[32]
Bne R1, R0, -3	[4]	[6, 6]	[]	[]	[33]
Addi R1, R1, -4	[7]	[8, 8]	[]	[9]	[33]
Addi R2, R2, -4	[7]	[8, 8]	[]	[10]	[34]
Bne R1, R0, -3	[8]	[10, 10]	[]	[]	[34]
Addi R1, R1, -4	[11]	[12, 12]	[]	[14]	[35]
Addi R2, R2, -4	[11]	[12, 12]	[]	[15]	[35]
Bne R1, R0, -3	[12]	[15, 15]	[]	[]	[36]
Add.d F20, F2, F2	[16]	[17, 20]	[]	[21]	[36]

Final non-zero registers: R2=20, F2=1.0, F4=3.0, F6=7.0, F20=2.0
Non-zero memory: Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=7.0

tc1 (given test case), issue width 3

	ISSUE	EX	MEM	WB	COMMIT
Ld F2, 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F4, F2, F20	[1]	[9, 23]	[]	[24]	[25]
Ld F6, 0(R2)	[1]	[2, 2]	[8, 12]	[13]	[25]
Add.d F6, F4, F6	[2]	[25, 28]	[]	[29]	[30]
Sd F6, 0(R2)	[2]	[3, 3]	[31, 35]	[]	[31]
Addi R1, R1, -4	[2]	[3, 3]	[]	[4]	[32]
Addi R2, R2, -4	[3]	[4, 4]	[]	[5]	[32]
Bne R1, R0, -3	[3]	[5, 5]	[]	[]	[32]
Addi R1, R1, -4	[6]	[7, 7]	[]	[9]	[33]
Addi R2, R2, -4	[6]	[7, 7]	[]	[10]	[33]
Bne R1, R0, -3	[6]	[10, 10]	[]	[]	[33]
Addi R1, R1, -4	[11]	[12, 12]	[]	[14]	[34]
Addi R2, R2, -4	[11]	[12, 12]	[]	[15]	[34]
Bne R1, R0, -3	[11]	[15, 15]	[]	[]	[34]
Add.d F20, F2, F2	[16]	[17, 20]	[]	[21]	[35]

Final non-zero registers: R2=20, F2=1.0, F4=3.0, F6=7.0, F20=2.0
Non-zero memory: Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=7.0

tc1 (given test case), issue width 4

	ISSUE	EX	MEM	WB	COMMIT
Ld F2, 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]
Mult.d F4, F2, F20	[1]	[9, 23]	[]	[24]	[25]
Ld F6, 0(R2)	[1]	[2, 2]	[8, 12]	[13]	[25]
Add.d F6, F4, F6	[1]	[25, 28]	[]	[29]	[30]
Sd F6, 0(R2)	[2]	[3, 3]	[31, 35]	[]	[31]
Addi R1, R1, -4	[2]	[3, 3]	[]	[4]	[32]
Addi R2, R2, -4	[2]	[3, 3]	[]	[5]	[32]
Bne R1, R0, -3	[2]	[5, 5]	[]	[]	[32]
Addi R1, R1, -4	[6]	[7, 7]	[]	[9]	[32]
Addi R2, R2, -4	[6]	[7, 7]	[]	[10]	[33]
Bne R1, R0, -3	[6]	[10, 10]	[]	[]	[33]
Addi R1, R1, -4	[11]	[12, 12]	[]	[14]	[33]
Addi R2, R2, -4	[11]	[12, 12]	[]	[15]	[33]
Bne R1, R0, -3	[11]	[15, 15]	[]	[]	[34]
Add.d F20, F2, F2	[16]	[17, 20]	[]	[21]	[34]

Final non-zero registers: R2=20, F2=1.0, F4=3.0, F6=7.0, F20=2.0
Non-zero memory: Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=7.0

4.3 Independent instructions: one CDB (tc2) and four CDBs (tc3)

tc2 (independent instructions, one CDB), issue width 1

	ISSUE	EX	MEM	WB	COMMIT
Add R4, R1, R2	[1]	[2, 2]	[]	[3]	[4]
Add.d F4, F1, F2	[2]	[3, 5]	[]	[6]	[7]
Mult.d F5, F2, F3	[3]	[4, 8]	[]	[9]	[10]
Sub R5, R3, R1	[4]	[5, 5]	[]	[7]	[11]
Addi R6, R2, 10	[5]	[6, 6]	[]	[8]	[12]
Sub.d F6, F3, F1	[6]	[7, 9]	[]	[10]	[13]
Mult.d F7, F1, F1	[7]	[8, 12]	[]	[13]	[14]
Add R7, R1, R3	[8]	[9, 9]	[]	[11]	[15]
Add.d F8, F2, F3	[9]	[10, 12]	[]	[14]	[16]
Addi R8, R3, -1	[10]	[11, 11]	[]	[12]	[17]
Mult.d F9, F3, F3	[11]	[12, 16]	[]	[17]	[18]
Sub R9, R2, R1	[12]	[13, 13]	[]	[15]	[19]

Final non-zero registers: R1=1, R2=2, R3=3, R4=3, R5=2, R6=12, R7=4, R8=2, R9=1, F1=1.5, F2=2.5, F3=4.0, F4=4.0, F5=10.0, F6=2.5, F7=2.25, F8=6.5, F9=16.0
Non-zero memory:

tc2 (independent instructions, one CDB), issue width 4

	ISSUE	EX	MEM	WB	COMMIT
Add R4, R1, R2	[1]	[2, 2]	[]	[3]	[4]
Add.d F4, F1, F2	[1]	[2, 4]	[]	[5]	[6]
Mult.d F5, F2, F3	[1]	[2, 6]	[]	[7]	[8]
Sub R5, R3, R1	[1]	[2, 2]	[]	[4]	[8]
Addi R6, R2, 10	[2]	[3, 3]	[]	[6]	[8]
Sub.d F6, F3, F1	[2]	[3, 5]	[]	[8]	[9]
Mult.d F7, F1, F1	[2]	[3, 7]	[]	[9]	[10]
Add R7, R1, R3	[2]	[3, 3]	[]	[10]	[11]
Add.d F8, F2, F3	[3]	[4, 6]	[]	[11]	[12]
Addi R8, R3, -1	[3]	[4, 4]	[]	[12]	[13]
Mult.d F9, F3, F3	[3]	[4, 8]	[]	[13]	[14]
Sub R9, R2, R1	[3]	[4, 4]	[]	[14]	[15]

Final non-zero registers: R1=1, R2=2, R3=3, R4=3, R5=2, R6=12, R7=4, R8=2, R9=1, F1=1.5, F2=2.5, F3=4.0, F4=4.0, F5=10.0, F6=2.5, F7=2.25, F8=6.5, F9=16.0
Non-zero memory:

tc3 (independent instructions, CDB = 4), issue width 1

	ISSUE	EX	MEM	WB	COMMIT
Add R4, R1, R2	[1]	[2, 2]	[]	[3]	[4]
Add.d F4, F1, F2	[2]	[3, 5]	[]	[6]	[7]
Mult.d F5, F2, F3	[3]	[4, 8]	[]	[9]	[10]
Sub R5, R3, R1	[4]	[5, 5]	[]	[6]	[11]
Addi R6, R2, 10	[5]	[6, 6]	[]	[7]	[12]
Sub.d F6, F3, F1	[6]	[7, 9]	[]	[10]	[13]
Mult.d F7, F1, F1	[7]	[8, 12]	[]	[13]	[14]
Add R7, R1, R3	[8]	[9, 9]	[]	[10]	[15]
Add.d F8, F2, F3	[9]	[10, 12]	[]	[13]	[16]
Addi R8, R3, -1	[10]	[11, 11]	[]	[12]	[17]
Mult.d F9, F3, F3	[11]	[12, 16]	[]	[17]	[18]
Sub R9, R2, R1	[12]	[13, 13]	[]	[14]	[19]

Final non-zero registers: R1=1, R2=2, R3=3, R4=3, R5=2, R6=12, R7=4, R8=2, R9=1, F1=1.5, F2=2.5, F3=4.0, F4=4.0, F5=10.0, F6=2.5, F7=2.25, F8=6.5, F9=16.0
Non-zero memory:

tc3 (independent instructions, CDB = 4), issue width 4

	ISSUE	EX	MEM	WB	COMMIT
Add R4, R1, R2	[1]	[2, 2]	[]	[3]	[4]
Add.d F4, F1, F2	[1]	[2, 4]	[]	[5]	[6]
Mult.d F5, F2, F3	[1]	[2, 6]	[]	[7]	[8]
Sub R5, R3, R1	[1]	[2, 2]	[]	[3]	[8]
Addi R6, R2, 10	[2]	[3, 3]	[]	[4]	[8]
Sub.d F6, F3, F1	[2]	[3, 5]	[]	[6]	[8]
Mult.d F7, F1, F1	[2]	[3, 7]	[]	[8]	[9]
Add R7, R1, R3	[2]	[3, 3]	[]	[4]	[9]
Add.d F8, F2, F3	[3]	[4, 6]	[]	[7]	[9]
Addi R8, R3, -1	[3]	[4, 4]	[]	[5]	[9]
Mult.d F9, F3, F3	[3]	[4, 8]	[]	[9]	[10]
Sub R9, R2, R1	[3]	[4, 4]	[]	[5]	[10]

Final non-zero registers: R1=1, R2=2, R3=3, R4=3, R5=2, R6=12, R7=4, R8=2, R9=1, F1=1.5, F2=2.5, F3=4.0, F4=4.0, F5=10.0, F6=2.5, F7=2.25, F8=6.5, F9=16.0
Non-zero memory:

4.4 Branches (tc7)

tc7 (branches), issue width 1

	ISSUE	EX	MEM	WB	COMMIT
Add.d F3, F3, F1	[1]	[2, 4]	[]	[5]	[6]
Mult.d F4, F3, F2	[2]	[6, 10]	[]	[11]	[12]
Addi R1, R1, 4	[3]	[4, 4]	[]	[6]	[13]
Sd F4, 0(R1)	[4]	[7, 7]	[14, 16]	[]	[14]
Bne R1, R2, -5	[5]	[7, 7]	[]	[]	[15]
Add.d F3, F3, F1	[8]	[9, 11]	[]	[12]	[16]
Mult.d F4, F3, F2	[9]	[13, 17]	[]	[18]	[19]
Addi R1, R1, 4	[10]	[11, 11]	[]	[13]	[20]
Sd F4, 0(R1)	[11]	[14, 14]	[21, 23]	[]	[21]
Bne R1, R2, -5	[12]	[14, 14]	[]	[]	[22]
Add.d F3, F3, F1	[15]	[16, 18]	[]	[19]	[23]
Mult.d F4, F3, F2	[16]	[20, 24]	[]	[25]	[26]
Addi R1, R1, 4	[17]	[18, 18]	[]	[20]	[27]
Sd F4, 0(R1)	[18]	[21, 21]	[28, 30]	[]	[28]
Bne R1, R2, -5	[19]	[21, 21]	[]	[]	[29]
Beq R1, R0, 2	[22]	[23, 23]	[]	[]	[30]
Addi R5, R0, 7	[24]	[25, 25]	[]	[26]	[31]
Add.d F5, F4, F4	[25]	[26, 28]	[]	[29]	[32]
Beq R5, R5, 1	[26]	[27, 27]	[]	[]	[33]
Addi R7, R5, 1	[28]	[29, 29]	[]	[30]	[34]

Final non-zero registers: R1=12, R2=12, R5=7, R7=8, F1=1.0, F2=0.5, F3=3.0, F4=1.5, F5=3.0
Non-zero memory: Mem[4]=0.5, Mem[8]=1.0, Mem[12]=1.5

tc7 (branches), issue width 4

	ISSUE	EX	MEM	WB	COMMIT
Add.d F3, F3, F1	[1]	[2, 4]	[]	[5]	[6]
Mult.d F4, F3, F2	[1]	[6, 10]	[]	[11]	[12]
Addi R1, R1, 4	[1]	[2, 2]	[]	[3]	[12]
Sd F4, 0(R1)	[1]	[4, 4]	[13, 15]	[]	[13]
Bne R1, R2, -5	[2]	[4, 4]	[]	[]	[14]
Add.d F3, F3, F1	[5]	[6, 8]	[]	[9]	[14]
Mult.d F4, F3, F2	[5]	[10, 14]	[]	[15]	[16]
Addi R1, R1, 4	[5]	[6, 6]	[]	[7]	[16]
Sd F4, 0(R1)	[5]	[8, 8]	[17, 19]	[]	[17]
Bne R1, R2, -5	[6]	[8, 8]	[]	[]	[18]
Add.d F3, F3, F1	[9]	[10, 12]	[]	[13]	[18]
Mult.d F4, F3, F2	[9]	[14, 18]	[]	[19]	[20]
Addi R1, R1, 4	[9]	[10, 10]	[]	[12]	[20]
Sd F4, 0(R1)	[9]	[13, 13]	[21, 23]	[]	[21]
Bne R1, R2, -5	[10]	[13, 13]	[]	[]	[22]
Beq R1, R0, 2	[14]	[15, 15]	[]	[]	[22]
Addi R5, R0, 7	[16]	[17, 17]	[]	[18]	[22]
Add.d F5, F4, F4	[16]	[20, 22]	[]	[23]	[24]
Beq R5, R5, 1	[16]	[19, 19]	[]	[]	[24]
Addi R7, R5, 1	[20]	[21, 21]	[]	[22]	[24]

Final non-zero registers: R1=12, R2=12, R5=7, R7=8, F1=1.0, F2=0.5, F3=3.0, F4=1.5, F5=3.0
Non-zero memory: Mem[4]=0.5, Mem[8]=1.0, Mem[12]=1.5

4.5 Memory disambiguation and forwarding (tc5)

A MEM entry with one cycle (for example `[16]`) means the load got its value from an older store in the load/store queue (forwarding). A MEM entry with two cycles is a memory access.

tc5 (memory), issue width 2

	ISSUE	EX	MEM	WB	COMMIT
Sd F1, 0(R1)	[1]	[2, 2]	[3, 5]	[]	[3]
Ld F3, 0(R1)	[1]	[2, 2]	[6, 8]	[9]	[10]
Ld F4, 0(R2)	[2]	[3, 3]	[9, 11]	[12]	[13]
Add.d F5, F3, F4	[2]	[13, 14]	[]	[15]	[16]
Sd F5, 8(R2)	[3]	[4, 4]	[17, 19]	[]	[17]
Ld F6, 24(R0)	[3]	[5, 5]	[16]	[17]	[18]
Addi R3, R1, 8	[4]	[5, 5]	[]	[6]	[18]
Sd F2, 0(R3)	[4]	[7, 7]	[20, 22]	[]	[20]
Ld F7, 16(R0)	[5]	[6, 6]	[8]	[10]	[21]
Mult.d F8, F6, F7	[5]	[18, 21]	[]	[22]	[23]
Sd F8, 32(R0)	[6]	[8, 8]	[24, 26]	[]	[24]
Ld F9, 32(R0)	[9]	[10, 10]	[23]	[24]	[25]

Final non-zero registers: R1=8, R2=16, R3=16, F1=1.5, F2=2.5, F3=1.5, F4=20.0, F5=21.5, F6=21.5, F7=2.5, F8=53.75, F9=53.75
Non-zero memory: Mem[8]=1.5, Mem[16]=2.5, Mem[24]=21.5, Mem[32]=53.75

The outputs of all 40 runs are in the `results/` folder.

5. What works and what does not work

Works

- Issue width 1–4 from the input file (default 1), in-order issue and in-order commit of up to W instructions per cycle, with renaming inside an issue group.
- Integer and FP instructions, `Ld`/`Sd`, `Bne`/`Beq`, register renaming, ROB, load/store queue with forwarding and disambiguation, several FUs per type, R0 = 0.
- Reading the configuration and program from the input file in the assignment format, and printing the status table, registers and non-zero memory.

- All 10 test cases give the same final state as the reference model, and follow the timing rules, at every issue width.

Limitations

- **No branch prediction.** The provided Python code does not implement the 1-bit predictor and BTB described in the assignment. It stops fetching after a branch until the branch is resolved in EX, and fetching continues in the next cycle. I kept this behavior (the task was to add multi-issue), so no instructions are squashed. This is why wide issue gains little in branch-heavy loops.
- **One CDB.** Write back is limited to one result per cycle, as specified. The optional `CDB = N` line is an extension for experiments only.
- **Input values.** Branch offsets must be given in instructions (as in the ISA of the assignment). Memory addresses must be inside the 256-byte memory.

6. How to run

```
cd code
python3 main.py ../tests/tc1_given_loop.txt      # width from the file
python3 main.py ../tests/tc1_given_loop.txt 3    # override the width
python3 run_tests.py                             # all tests, widths 1-4, with checks
```

Appendix A. Details of the changes

A.1 Input file and output – `init.py`, `main.py`

- `read_input()` (new, in `init.py`) reads the configuration table (`# of rs`, `Cycles in EX`, `Cycles in Mem`, `# of FUs`), `ROB entries`, **`Issue width`** (1–4, default 1), the register line (`R1=10, F2=30.1`), the memory line (`Mem[4]=1`) and the program. The text is case insensitive and operands can be separated by commas and/or spaces (`Ld F4, 8(R1)` or `ld f4 8(r1)`). Before, all of this was hard-coded in `main.py`.
- `main.py` builds the machine from this configuration, passes the issue width to the issue and commit stages, and prints the instruction status table, all registers and the non-zero memory values. Usage: `python3 main.py <input file> [issue width]`. The optional second argument overrides the width in the file, which makes it easy to run the same program with widths 1–4.
- `# of FUs` is now used. The FP adder and multiplier are pipelined, so `N` FUs accept up to `N` new instructions per cycle. The integer adder is unpipelined, and the load/store unit gets `N` address adders. Memory stays single-ported.
- `Beq` was added (the original code only had `Bne`). Branch targets follow the ISA of the assignment: `PC + 4 + offset<<2`, so the offset counts instructions. The provided test case was written for the old byte offset (`Bne R1 R0 -12`); in my test files it is `Bne R1, R0, -3`, which is the same target.
- Optional `CDB = N` line (default 1, as specified). I only use it in one test case to show how much the single CDB limits a wide machine.

A.2 Commit stage – `commit.py`

`commit()` now removes up to `issue_width` finished instructions from the head of the ROB per cycle, in order, and stops at the first instruction that is not finished. Branches commit after they are resolved and now also get a commit cycle in the table. Stores keep the original rule:

a store commits when it starts its memory access. The table rows are collected and printed at the end.

A.3 Bugs fixed in the provided code

These problems were in the original code. Most of them only appear when several instructions are in flight, so multi-issue made them visible:

File	Problem	Fix
`wb.py`	A broadcast always wrote its value into the RAT. If a younger instruction had already renamed the same register (WAW), the RAT lost the younger tag and later instructions read a stale value.	The RAT is updated only if it still holds the tag of the broadcasting instruction.
`exe.py`	`fu_exe()` was called a second time in the same cycle for a newly dispatched instruction, which advanced every instruction already in that FU twice.	Only the new instruction is advanced in the second call.
`exe.py`	Address calculation never finished if it took more than 1 cycle.	The counter is advanced every cycle.
`exe.py`	Ready instructions were dispatched by reservation-station index, not by age.	Oldest ready instruction first.
`mem.py`	`check_if_sd_committable` was used without being called (always true, and it had a wrong field name), so stores could write memory before older instructions committed.	A store writes memory only when it is at the head of the ROB, i.e. when it commits.
`mem.py`	Loads were blocked whenever the first queue entry was a store with data (checked `ld_sd_queue[0]` instead of the load itself).	Check the load's own entry.
`mem.py`	Forwarding only looked at the nearest store, and the loop variable was overwritten inside the loop.	Look back from the nearest store; stop at a store with an unknown address; forward from the first store with the same address. Forwarding happens one cycle after the load's address is calculated.
`wb.py`	With one CDB, results were broadcast in completion order, so a younger instruction could delay the ROB head.	Oldest instruction first.
`main.py`	The simulation could stop before the last store reached memory, or while instructions were still waiting to be fetched after a branch.	Run until the ROB, the program and the load/store unit are all finished.
`issue.py`	Writing R0 renamed R0; load/store parsing only worked for `R0`–`R9`; tag choice was random.	R0 is never renamed (always 0); proper `offset(Rn)` parsing; smallest free tag.

A.4 Timing conventions

These are the same as in the original code:

- An instruction issued in cycle `c` can start EX in cycle `c+1`.
- Its result is written back on the CDB at the earliest one cycle after EX (or MEM for loads) finishes.
- It commits at the earliest one cycle after write back.
- A load's memory access starts the cycle after its address is calculated.
- A store commits in the cycle it starts its memory access.

With issue width 1 the simulator behaves like the original single-issue code (except for the bug fixes and the commit cycle now shown for branches).